

UNIVERSIDAD NACIONAL DE COLOMBIA

Programación Orientada a Objetos 2026-2

Actividad 2

Ejercicios de clases y objetos

Libro: Ejercicios de programación orientada a objetos con Java y UML

(Leonardo Bermón Angarita)

Estudiante: Carlos Julio Nieto Morales

Docente: Walter Hugo Arboleda Mazo

Octubre de 2026

Introducción

Este documento presenta la solución de los ejercicios 2.1 a 2.5 del libro Ejercicios de programación orientada a objetos con Java y UML, de Leonardo Bermón Angarita, correspondientes a la Actividad 2 de la asignatura Programación Orientada a Objetos. Los ejercicios se desarrollaron en el lenguaje Python.

Para cada ejercicio se incluyen una breve descripción de la solución, el diagrama de clases, el código fuente, el resultado de la ejecución del programa y la dirección del repositorio en GitHub. Cada ejercicio incorpora también los ejercicios propuestos que el libro plantea al final del tema correspondiente.

Ejercicio 2.1. Definición de una clase

Descripción de la solución

Se define la clase Persona con los atributos nombre, apellidos, número de documento de identidad y año de nacimiento, junto con un constructor que inicializa sus valores y un método imprimir que los muestra en pantalla.

Siguiendo los ejercicios propuestos, se agregaron dos atributos: el país de nacimiento (cadena de texto) y el género, que se representa con un solo carácter, 'H' o 'M'. Como Python no tiene un tipo char, el género se guarda como una cadena de un carácter y el constructor valida que sea 'H' o 'M'. Además, se modificaron el constructor y el método imprimir para que incluyan los nuevos atributos.

Diagrama de clases

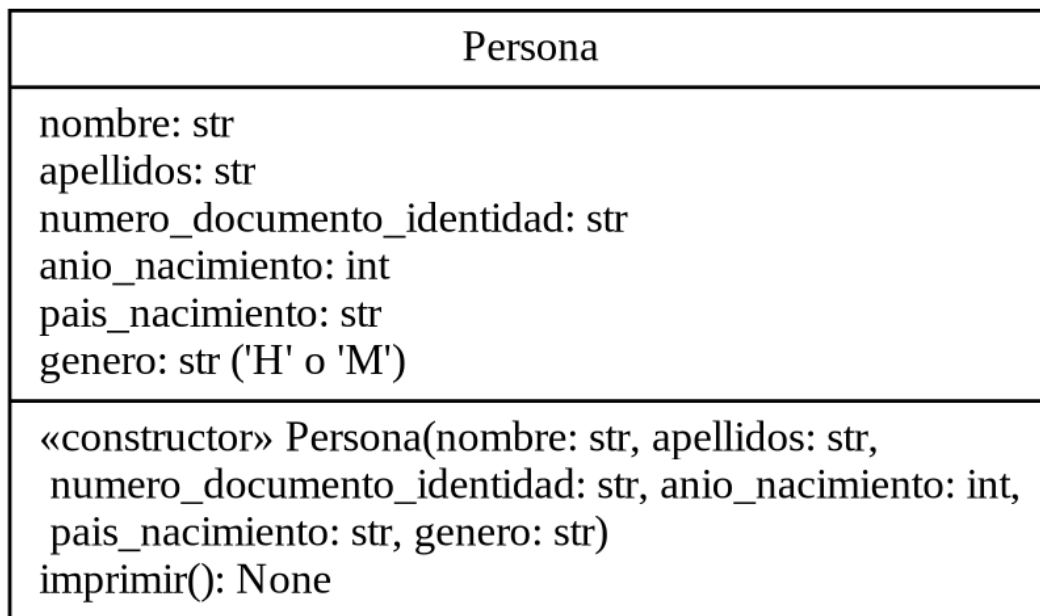


Figura 1. Diagrama de clases del ejercicio 2.1.

Código fuente

```
class Persona:
    def __init__(self, nombre: str, apellidos: str, numero_documento_identidad: str,
                  anio_nacimiento: int, pais_nacimiento: str, genero: str):
        if genero not in ("H", "M"):
            raise ValueError("El género debe ser 'H' o 'M'.")
        self.nombre = nombre
        self.apellidos = apellidos
        self.numero_documento_identidad = numero_documento_identidad
        self.anio_nacimiento = anio_nacimiento
        self.pais_nacimiento = pais_nacimiento
        self.genero = genero

    def imprimir(self) :
        print(f'Nombre = {self.nombre}')
        print(f'Apellidos = {self.apellidos}')
        print(f'Número de documento de identidad = {self.numero_documento_identidad}')
        print(f'Año de nacimiento = {self.anio_nacimiento}')
        print(f'País de nacimiento = {self.pais_nacimiento}')
        print(f'Género = {self.genero}')

def main() :
    p1 = Persona("Pedro", "Pérez", "1053121010", 1998, "Colombia", "H")
    p2 = Persona("Luis", "León", "1053223344", 2001, "Colombia", "H")
    p1.imprimir()
    print()
    p2.imprimir()

if __name__ == "__main__":
    main()
```

Ejecución del programa

Nombre = Pedro
Apellidos = Pérez
Número de documento de identidad = 1053121010
Año de nacimiento = 1998
País de nacimiento = Colombia
Género = H

Nombre = Luis
Apellidos = León
Número de documento de identidad = 1053223344
Año de nacimiento = 2001
País de nacimiento = Colombia
Género = H

Ejercicio 2.2. Definición de atributos de una clase con tipos primitivos de datos

Descripción de la solución

Se define la clase Planeta con los atributos nombre, cantidad de satélites, masa, volumen, diámetro, distancia al Sol, tipo de planeta y si es observable. El tipo de planeta se modela con una enumeración (TipoPlaneta) con los valores GASEOSO, TERRESTRE y ENANO, y se relaciona con la clase Planeta mediante una asociación.

La clase incluye el método calcular_densidad, que divide la masa entre el volumen, y el método es_planeta_exterior, que devuelve verdadero cuando la distancia al Sol supera 3,4 unidades astronómicas (1 UA equivale a 149 597 870 km); este umbral es el criterio adoptado en la solución. Como ejercicio propuesto se agregaron los atributos periodo orbital (en años) y periodo de rotación (en días), que se inicializan en el constructor y se muestran en el método imprimir.

Diagrama de clases

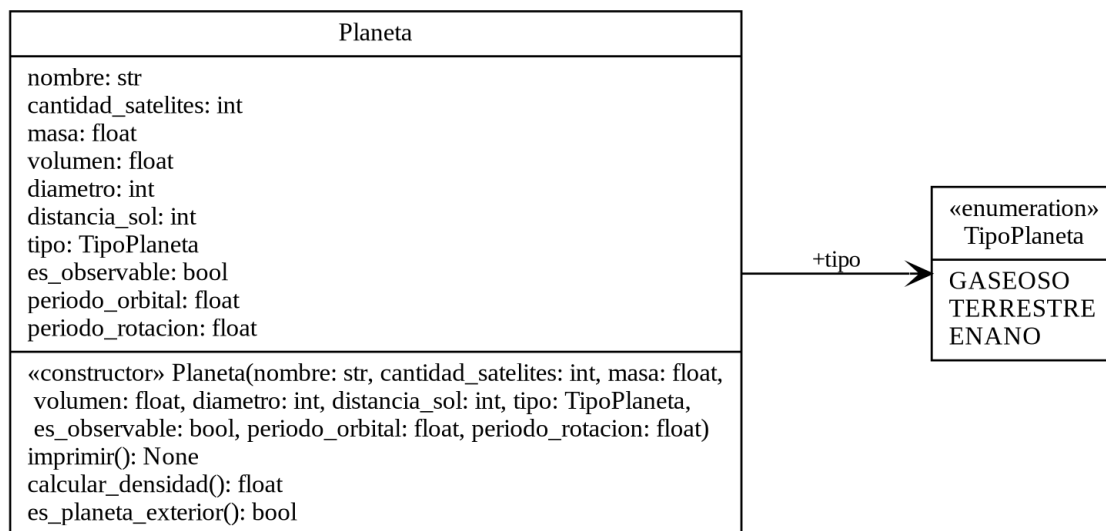


Figura 2. Diagrama de clases del ejercicio 2.2.

Código fuente

```
from enum import Enum

class TipoPlaneta(Enum):
    GASEOSO = "GASEOSO"
    TERRESTRE = "TERRESTRE"
    ENANO = "ENANO"

class Planeta:
    # 1 UA = 149 597 870 km. Se considera exterior un planeta más allá de 3,4 UA.
    UNIDAD_ASTRONOMICA_KM = 149_597_870
    LIMITE_EXTERIOR_UA = 3.4

    def __init__(self, nombre: str, cantidad_satelites: int, masa: float,
                  volumen: float, diametro: int, distancia_sol: int,
                  tipo: TipoPlaneta, es_observable: bool,
                  periodo_orbital: float, periodo_rotacion: float):
        self.nombre = nombre
        self.cantidad_satelites = cantidad_satelites
        self.masa = masa # kg
        self.volumen = volumen # km^3
        self.diametro = diametro # km
        self.distancia_sol = distancia_sol # km
        self.tipo = tipo
        self.es_observable = es_observable
        self.periodo_orbital = periodo_orbital # años
        self.periodo_rotacion = periodo_rotacion # días

    def imprimir(self):
        print(f"Nombre del planeta = {self.nombre}")
        print(f"Cantidad de satélites = {self.cantidad_satelites}")
        print(f"Masa del planeta = {self.masa}")
        print(f"Volumen del planeta = {self.volumen}")
        print(f"Diámetro del planeta = {self.diametro}")
        print(f"Distancia al sol = {self.distancia_sol}")
        print(f"Tipo de planeta = {self.tipo.name}")
        print(f"Es observable = {str(self.es_observable).lower()}")
        print(f"Periodo orbital (años) = {self.periodo_orbital}")
        print(f"Periodo de rotación (días) = {self.periodo_rotacion}")
        print(f"Densidad del planeta = {self.calcular_densidad()}")
        print(f"Es planeta exterior = {str(self.es_planeta_exterior()).lower()}")

    def calcular_densidad(self):
        return self.masa / self.volumen

    def es_planeta_exterior(self):
        limite = self.LIMITE_EXTERIOR_UA * self.UNIDAD_ASTRONOMICA_KM
        return self.distancia_sol > limite

def main():
    tierra = Planeta("Tierra", 1, 5.9736e24, 1.08321e12, 12742, 150000000,
                    TipoPlaneta.TERRESTRE, True, 1.0, 1.0)
    jupiter = Planeta("Júpiter", 79, 1.899e27, 1.4313e15, 139820, 750000000,
```

```
TipoPlaneta.GASEOSO, True, 11.86, 0.41)
tierra.imprimir()
print()
jupiter.imprimir()
```

```
if __name__ == "__main__":
    main()
```

Ejecución del programa

Nombre del planeta = Tierra
Cantidad de satélites = 1
Masa del planeta = 5.9736e+24
Volumen del planeta = 1083210000000.0
Diámetro del planeta = 12742
Distancia al sol = 150000000
Tipo de planeta = TERRESTRE
Es observable = true
Periodo orbital (años) = 1.0
Periodo de rotación (días) = 1.0
Densidad del planeta = 5514720137369.484
Es planeta exterior = false

Nombre del planeta = Júpiter
Cantidad de satélites = 79
Masa del planeta = 1.899e+27
Volumen del planeta = 1431300000000000.0
Diámetro del planeta = 139820
Distancia al sol = 750000000
Tipo de planeta = GASEOSO
Es observable = true
Periodo orbital (años) = 11.86
Periodo de rotación (días) = 0.41
Densidad del planeta = 1326765877174.5964
Es planeta exterior = true

Ejercicio 2.3. Estado de un objeto

Descripción de la solución

Se define la clase Automóvil con los atributos marca, modelo, motor, tipo de combustible, tipo de automóvil, número de puertas, cantidad de asientos, velocidad máxima, color y velocidad actual. Los tipos de combustible, de automóvil y de color se modelan como enumeraciones asociadas a la clase.

Los métodos get y set de cada atributo se implementan como propiedades de Python (@property y su respectivo setter). La clase incluye además los métodos acelerar, que no permite superar la velocidad máxima; desacelerar, que no permite una velocidad negativa; frenar, que lleva la velocidad actual a cero; calcular_tiempo_llegada, que devuelve las horas necesarias para recorrer una distancia a la velocidad actual; e imprimir.

Diagrama de clases

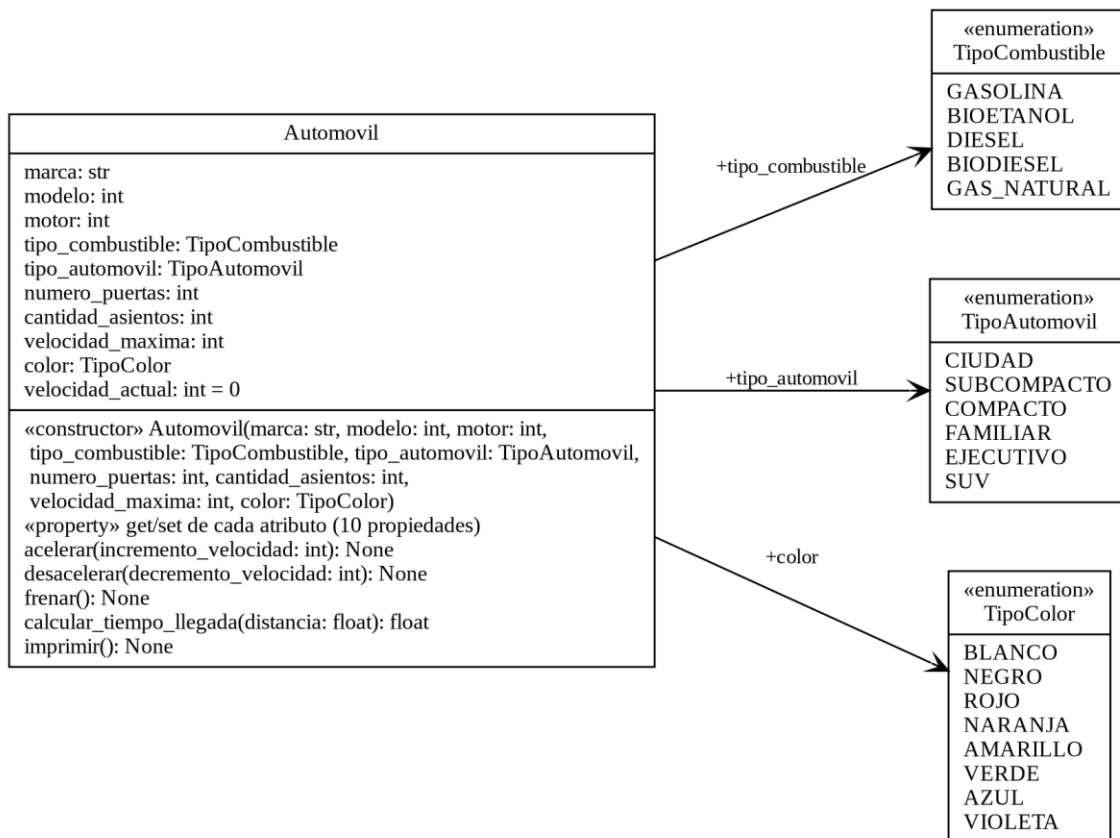


Figura 3. Diagrama de clases del ejercicio 2.3.

Código fuente

```
from enum import Enum
```

```
class TipoCombustible(Enum):  
    GASOLINA = "GASOLINA"  
    BIOETANOL = "BIOETANOL"  
    DIESEL = "DIESEL"  
    BIODIESEL = "BIODIESEL"  
    GAS_NATURAL = "GAS_NATURAL"
```

```
class TipoAutomovil(Enum):  
    CIUDAD = "CIUDAD"  
    SUBCOMPACTO = "SUBCOMPACTO"  
    COMPACTO = "COMPACTO"  
    FAMILIAR = "FAMILIAR"  
    EJECUTIVO = "EJECUTIVO"  
    SUV = "SUV"
```

```
class TipoColor(Enum):  
    BLANCO = "BLANCO"  
    NEGRO = "NEGRO"  
    ROJO = "ROJO"  
    NARANJA = "NARANJA"  
    AMARILLO = "AMARILLO"  
    VERDE = "VERDE"  
    AZUL = "AZUL"  
    VIOLETA = "VIOLETA"
```

```
class Automovil:  
    def __init__(self, marca: str, modelo: int, motor: int,  
                 tipo_combustible: TipoCombustible, tipo_automovil: TipoAutomovil,  
                 numero_puertas: int, cantidad_asientos: int,  
                 velocidad_maxima: int, color: TipoColor):  
        self._marca = marca  
        self._modelo = modelo  
        self._motor = motor  
        self._tipo_combustible = tipo_combustible  
        self._tipo_automovil = tipo_automovil  
        self._numero_puertas = numero_puertas  
        self._cantidad_asientos = cantidad_asientos  
        self._velocidad_maxima = velocidad_maxima  
        self._color = color  
        self._velocidad_actual = 0
```

```
@property  
def marca(self) :  
    return self._marca
```

```
@marca.setter  
def marca(self, marca: str) :  
    self._marca = marca
```

```
@property
```

```

def modelo(self) :
    return self._modelo

@modelo.setter
def modelo(self, modelo: int) :
    self._modelo = modelo

@property
def motor(self) :
    return self._motor

@motor.setter
def motor(self, motor: int) :
    self._motor = motor

@property
def tipo_combustible(self):
    return self._tipo_combustible

@tipo_combustible.setter
def tipo_combustible(self, tipo_combustible: TipoCombustible) :
    self._tipo_combustible = tipo_combustible

@property
def tipo_automovil(self):
    return self._tipo_automovil

@tipo_automovil.setter
def tipo_automovil(self, tipo_automovil: TipoAutomovil) :
    self._tipo_automovil = tipo_automovil

@property
def numero_puertas(self) :
    return self._numero_puertas

@numero_puertas.setter
def numero_puertas(self, numero_puertas: int) :
    self._numero_puertas = numero_puertas

@property
def cantidad_asientos(self) :
    return self._cantidad_asientos

@cantidad_asientos.setter
def cantidad_asientos(self, cantidad_asientos: int) :
    self._cantidad_asientos = cantidad_asientos

@property
def velocidad_maxima(self) :
    return self._velocidad_maxima

@velocidad_maxima.setter
def velocidad_maxima(self, velocidad_maxima: int) :
    self._velocidad_maxima = velocidad_maxima

@property
def color(self):

```

```

        return self._color

    @color.setter
    def color(self, color: TipoColor) :
        self._color = color

    @property
    def velocidad_actual(self) :
        return self._velocidad_actual

    @velocidad_actual.setter
    def velocidad_actual(self, velocidad_actual: int) :
        self._velocidad_actual = velocidad_actual

    def acelerar(self, incremento_velocidad: int) :
        nueva = self._velocidad_actual + incremento_velocidad
        if nueva > self._velocidad_maxima:
            print("No se puede superar la velocidad máxima.")
            self._velocidad_actual = self._velocidad_maxima
        else:
            self._velocidad_actual = nueva

    def desacelerar(self, decremento_velocidad: int) :
        nueva = self._velocidad_actual - decremento_velocidad
        if nueva < 0:
            print("No se puede decrementar a una velocidad negativa.")
        else:
            self._velocidad_actual = nueva

    def frenar(self) :
        self._velocidad_actual = 0

    def calcular_tiempo_llegada(self, distancia: float) :
        """Tiempo en horas para recorrer 'distancia' (km) a la velocidad actual."""
        if self._velocidad_actual <= 0:
            raise ValueError("El automóvil está detenido; no se puede calcular el tiempo.")
        return distancia / self._velocidad_actual

    def imprimir(self) :
        print(f"Marca = {self._marca}")
        print(f"Modelo = {self._modelo}")
        print(f"Motor = {self._motor}")
        print(f"Tipo de combustible = {self._tipo_combustible.name}")
        print(f"Tipo de automóvil = {self._tipo_automovil.name}")
        print(f"Número de puertas = {self._numero_puertas}")
        print(f"Cantidad de asientos = {self._cantidad_asientos}")
        print(f"Velocidad máxima = {self._velocidad_maxima}")
        print(f"Color = {self._color.name}")
        print(f"Velocidad actual = {self._velocidad_actual}")

def main() :
    auto1 = Automovil("Ford", 2018, 3, TipoCombustible.DIESEL,
                      TipoAutomovil.EJECUTIVO, 5, 6, 250, TipoColor.NEGRO)
    auto1.imprimir()
    auto1.acelerar(100)
    print(f"Velocidad actual = {auto1.velocidad_actual}")

```

```
auto1.acelerar(20)
print(f"Velocidad actual = {auto1.velocidad_actual}")
auto1.desacelerar(50)
print(f"Velocidad actual = {auto1.velocidad_actual}")
print(f"Tiempo de llegada para 140 km = {auto1.calcular_tiempo_llegada(140)} horas")
auto1.frenar()
print(f"Velocidad actual = {auto1.velocidad_actual}")
auto1.desacelerar(10)
```

```
if __name__ == "__main__":
    main()
```

Ejecución del programa

Marca = Ford
Modelo = 2018
Motor = 3
Tipo de combustible = DIESEL
Tipo de automóvil = EJECUTIVO
Número de puertas = 5
Cantidad de asientos = 6
Velocidad máxima = 250
Color = NEGRO
Velocidad actual = 0
Velocidad actual = 100
Velocidad actual = 120
Velocidad actual = 70
Tiempo de llegada para 140 km = 2.0 horas
Velocidad actual = 0
No se puede decrementar a una velocidad negativa.

Ejercicio 2.4. Figuras geométricas

Descripción de la solución

Se define una clase para cada figura geométrica (Círculo, Cuadrado, Rectángulo y TriánguloRectángulo), cada una con los métodos para calcular el área y el perímetro. La clase TriánguloRectángulo incluye además el cálculo de la hipotenusa y un método que determina si el triángulo es equilátero, isósceles o escaleno según la longitud de sus tres lados. La clase PruebaFiguras contiene el método principal que crea los objetos y muestra los resultados.

Como ejercicios propuestos se agregaron las clases Rombo, que recibe sus diagonales mayor y menor, y Trapecio, que recibe sus bases, su altura y la longitud de los dos lados no paralelos. Para el triángulo de base 3 y altura 5, el área se calcula como base por altura sobre dos, es decir 7,5.

Diagrama de clases

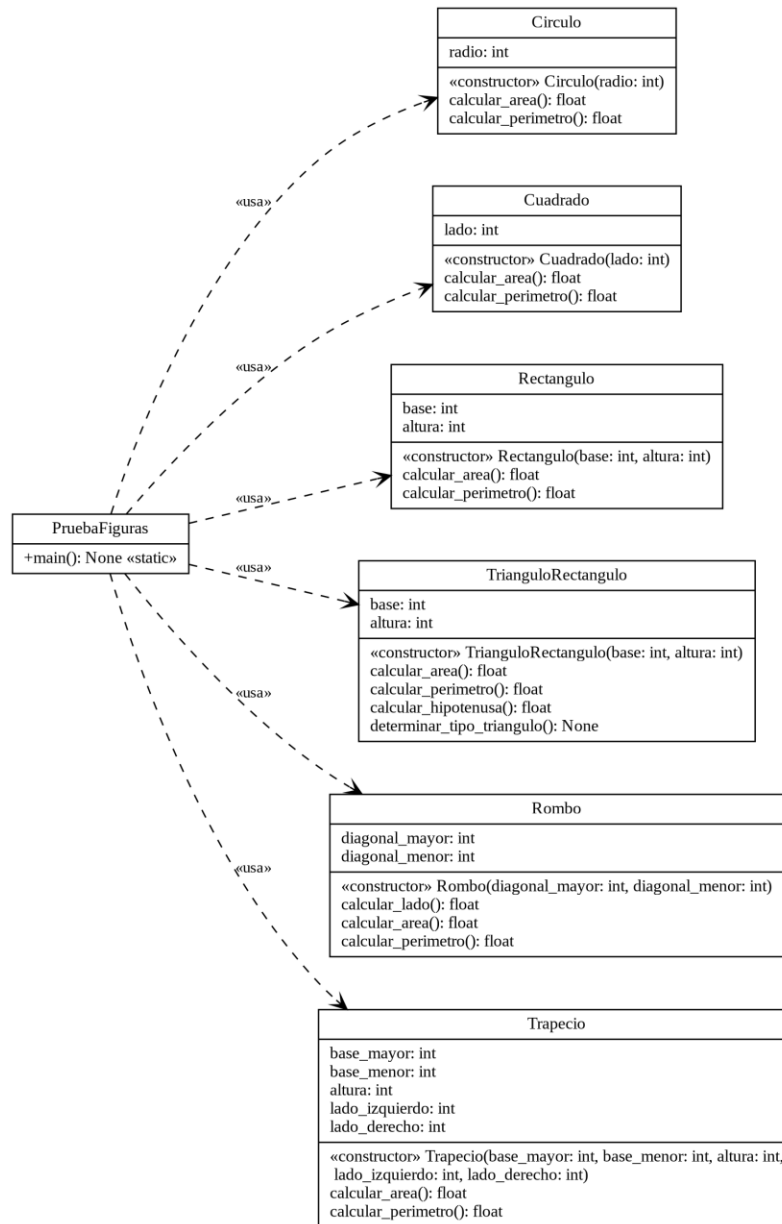


Figura 4. Diagrama de clases del ejercicio 2.4.

Código fuente

```
import math
```

```
class Circulo:
    def __init__(self, radio: int):
        self.radio = radio
```

```

def calcular_area(self) :
    return math.pi * self.radio ** 2

def calcular_perimetro(self) :
    return 2 * math.pi * self.radio

class Cuadrado:
    def __init__(self, lado: int):
        self.lado = lado

    def calcular_area(self) :
        return float(self.lado ** 2)

    def calcular_perimetro(self) :
        return float(4 * self.lado)

class Rectangulo:
    def __init__(self, base: int, altura: int):
        self.base = base
        self.altura = altura

    def calcular_area(self) :
        return float(self.base * self.altura)

    def calcular_perimetro(self) :
        return float(2 * (self.base + self.altura))

class TrianguloRectangulo:
    def __init__(self, base: int, altura: int):
        self.base = base
        self.altura = altura

    def calcular_area(self) :
        return self.base * self.altura / 2

    def calcular_hipotenusa(self) :
        return math.hypot(self.base, self.altura)

    def calcular_perimetro(self) :
        return self.base + self.altura + self.calcular_hipotenusa()

    def determinar_tipo_triangulo(self) :

        h = self.calcular_hipotenusa()
        lados = [self.base, self.altura, h]
        iguales = len({round(l, 9) for l in lados})
        if iguales == 1:
            print("Es un triángulo equilátero")
        elif iguales == 2:
            print("Es un triángulo isósceles")
        else:
            print("Es un triángulo escaleno")

```

```

class Rombo:
    def __init__(self, diagonal_mayor: int, diagonal_menor: int):
        self.diagonal_mayor = diagonal_mayor
        self.diagonal_menor = diagonal_menor

    def calcular_lado(self) :
        return math.hypot(self.diagonal_mayor / 2, self.diagonal_menor / 2)

    def calcular_area(self) :
        return self.diagonal_mayor * self.diagonal_menor / 2

    def calcular_perimetro(self) :
        return 4 * self.calcular_lado()

class Trapecio:
    def __init__(self, base_mayor: int, base_menor: int, altura: int,
        lado_izquierdo: int, lado_derecho: int):
        self.base_mayor = base_mayor
        self.base_menor = base_menor
        self.altura = altura
        self.lado_izquierdo = lado_izquierdo
        self.lado_derecho = lado_derecho

    def calcular_area(self) :
        return (self.base_mayor + self.base_menor) * self.altura / 2

    def calcular_perimetro(self) :
        return float(self.base_mayor + self.base_menor
            + self.lado_izquierdo + self.lado_derecho)

class PruebaFiguras:
    @staticmethod
    def main() :
        figura1 = Circulo(2)
        figura2 = Rectangulo(1, 2)
        figura3 = Cuadrado(3)
        figura4 = TrianguloRectangulo(3, 5)
        figura5 = Rombo(8, 6)
        figura6 = Trapecio(10, 6, 4, 5, 5)

        print(f"El área del círculo es = {figura1.calcular_area()}")
        print(f"El perímetro del círculo es = {figura1.calcular_perimetro()}")
        print()
        print(f"El área del rectángulo es = {figura2.calcular_area()}")
        print(f"El perímetro del rectángulo es = {figura2.calcular_perimetro()}")
        print()
        print(f"El área del cuadrado es = {figura3.calcular_area()}")
        print(f"El perímetro del cuadrado es = {figura3.calcular_perimetro()}")
        print()
        print(f"El área del triángulo es = {figura4.calcular_area()}")
        print(f"El perímetro del triángulo es = {figura4.calcular_perimetro()}")
        figura4.determinar_tipo_triangulo()
        print()
        print(f"El área del rombo es = {figura5.calcular_area()}")

```

```
print(f"El perímetro del rombo es = {figura5.calcular_perimetro()}")
print()
print(f"El área del trapecio es = {figura6.calcular_area()}")
print(f"El perímetro del trapecio es = {figura6.calcular_perimetro()}")
```

```
if __name__ == "__main__":
    PruebaFiguras.main()
```

Ejecución del programa

El área del círculo es = 12.566370614359172
El perímetro del círculo es = 12.566370614359172

El área del rectángulo es = 2.0
El perímetro del rectángulo es = 6.0

El área del cuadrado es = 9.0
El perímetro del cuadrado es = 12.0

El área del triángulo es = 7.5
El perímetro del triángulo es = 13.8309518948453
Es un triángulo escaleno

El área del rombo es = 24.0
El perímetro del rombo es = 20.0

El área del trapecio es = 32.0
El perímetro del trapecio es = 26.0

Ejercicio 2.5. Definición de métodos con parámetros

Descripción de la solución

Se define la clase CuentaBancaria con los nombres y apellidos del titular, el número de cuenta, el tipo de cuenta (enumeración TipoCuenta con los valores AHORROS y CORRIENTE) y el saldo, que inicia en cero. Los métodos consignar y retirar reciben el valor de la operación y devuelven un valor booleano que indica si se pudo realizar.

Los métodos comparar_cuentas y transferencia reciben otra cuenta como parámetro: el primero compara los saldos de ambas cuentas y el segundo traslada un valor desde la cuenta actual hacia la otra. Como ejercicios propuestos se agregó el atributo interés mensual (expresado como porcentaje) y el método calcular_nuevo_saldo, que aplica ese interés al saldo.

Diagrama de clases

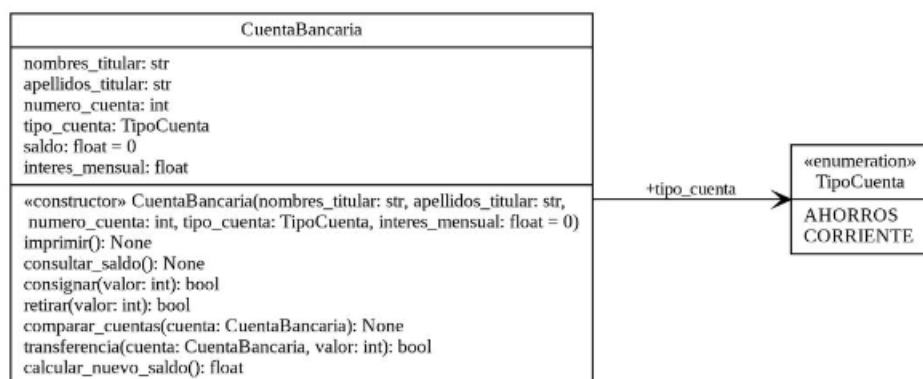


Figura 5. Diagrama de clases del ejercicio 2.5.

Código fuente

```
from enum import Enum
```

```
class TipoCuenta(Enum):
    AHORROS = "AHORROS"
    CORRIENTE = "CORRIENTE"
```

```
class CuentaBancaria:
    def __init__(self, nombres_titular: str, apellidos_titular: str,
                 numero_cuenta: int, tipo_cuenta: TipoCuenta,
                 interes_mensual: float = 0.0):
```

```

self.nombres_titular = nombres_titular
self.apellidos_titular = apellidos_titular
self.numero_cuenta = numero_cuenta
self.tipo_cuenta = tipo_cuenta
self.saldo = 0.0
self.interes_mensual = interes_mensual

def imprimir(self) :
    print(f"Nombres del titular = {self.nombres_titular}")
    print(f"Apellidos del titular = {self.apellidos_titular}")
    print(f"Número de cuenta = {self.numero_cuenta}")
    print(f"Tipo de cuenta = {self.tipo_cuenta.name}")
    print(f"Saldo = {self.saldo}")
    print(f"Interés mensual = {self.interes_mensual} %")

def consultar_saldo(self) :
    print(f"El saldo actual de la cuenta {self.numero_cuenta} es ${self.saldo}")

def consignar(self, valor: int) :
    if valor <= 0:
        print("El valor a consignar debe ser mayor que cero.")
        return False
    self.saldo += valor
    print(f"Se ha consignado ${valor} en la cuenta. El nuevo saldo es ${self.saldo}")
    return True

def retirar(self, valor: int) :
    if valor <= 0:
        print("El valor a retirar debe ser mayor que cero.")
        return False
    if valor > self.saldo:
        print("Saldo insuficiente para realizar el retiro.")
        return False
    self.saldo -= valor
    print(f"Se ha retirado ${valor} en la cuenta. El nuevo saldo es ${self.saldo}")
    return True

def comparar_cuentas(self, cuenta: "CuentaBancaria") :

    if self.saldo > cuenta.saldo:
        print(f"La cuenta {self.numero_cuenta} tiene mayor saldo que la cuenta {cuenta.numero_cuenta}.")
    elif self.saldo < cuenta.saldo:
        print(f"La cuenta {cuenta.numero_cuenta} tiene mayor saldo que la cuenta {self.numero_cuenta}.")
    else:
        print("Ambas cuentas tienen el mismo saldo.")

def transferencia(self, cuenta: "CuentaBancaria", valor: int) :

    if valor <= 0 or valor > self.saldo:
        print("No se pudo realizar la transferencia.")
        return False
    self.saldo -= valor
    cuenta.saldo += valor
    print(f"Se han transferido ${valor} de la cuenta {self.numero_cuenta} "
          f"a la cuenta {cuenta.numero_cuenta}.")
    return True

def calcular_nuevo_saldo(self) :

```

```

self.saldo += self.saldo * self.interes_mensual / 100
print(f"Se aplicó un interés del {self.interes_mensual} %. El nuevo saldo es ${self.saldo}")
return self.saldo

```

```

def main() :
    cuenta = CuentaBancaria("Pedro", "Pérez", 123456789, TipoCuenta.AHORROS, 1.5)
    cuenta.imprimir()
    cuenta.consignar(200000)
    cuenta.consignar(300000)
    cuenta.retirar(400000)
    cuenta.calcular_nuevo_saldo()

    print()
    otra = CuentaBancaria("Luis", "León", 987654321, TipoCuenta.CORRIENTE)
    otra.consignar(50000)
    cuenta.comparar_cuentas(otra)
    cuenta.transferencia(otra, 60000)
    cuenta.consultar_saldo()
    otra.consultar_saldo()

```

```

if __name__ == "__main__":
    main()

```

Ejecución del programa

Nombres del titular = Pedro
 Apellidos del titular = Pérez
 Número de cuenta = 123456789
 Tipo de cuenta = AHORROS
 Saldo = 0.0
 Interés mensual = 1.5 %
 Se ha consignado \$200000 en la cuenta. El nuevo saldo es \$200000.0
 Se ha consignado \$300000 en la cuenta. El nuevo saldo es \$500000.0
 Se ha retirado \$400000 en la cuenta. El nuevo saldo es \$100000.0
 Se aplicó un interés del 1.5 %. El nuevo saldo es \$101500.0

Se ha consignado \$50000 en la cuenta. El nuevo saldo es \$50000.0
 La cuenta 123456789 tiene mayor saldo que la cuenta 987654321.
 Se han transferido \$60000 de la cuenta 123456789 a la cuenta 987654321.
 El saldo actual de la cuenta 123456789 es \$41500.0
 El saldo actual de la cuenta 987654321 es \$110000.0

Referencia

Bermón Angarita, L. Ejercicios de programación orientada a objetos con Java y UML.

Enlace en GitHub

URL del ejercicio en GitHub: <https://github.com/CarlosJulioNietoMorales/POO-2026-02>